UNIVERSIDADE DE MOGI DAS CRUZES

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

LETICIA GABRIELA FERREIRA

THINDESK

Mogi das Cruzes

2026

LETICIA GABRIELA FERREIRA

THINDESK

Trabalho desenvolvido para apresentação à disciplina de Aplicativos Web, do curso de Sistemas de Informação da Universidade de Mogi das Cruzes como requisito parcial para aprovação na disciplina.

Professor: Alessandro Horas

Mogi das Cruzes

2026

Sumário

1 INTRODUÇÃO....4

2 REQUISITOS FUNCIONAIS....4

2.1 Requisitos funcionais de usuário (por tela)....4

2.2 Requisitos funcionais de sistema....5

3 REQUISITOS NÃO FUNCIONAIS....6

4 MODELAGEM DE DADOS....7

5 DOCUMENTAÇÃO DA API....7

5.1 Autenticação....7

5.2 Endpoints....7

5.3 Parâmetros de filtro....8

5.4 Exemplos de requisição e resposta....8

5.5 Possíveis erros....10

6 DASHBOARD ADMINISTRATIVO....10

7 EVIDÊNCIAS DE TESTES....11

8 ADAPTAÇÃO PARA SISTEMA DE PIZZARIA....12

9 CONSIDERAÇÕES FINAIS....13

REFERÊNCIAS....14

1 INTRODUÇÃO

Este documento apresenta o Dashboard Administrativo de chamados técnicos, Thindesk, desenvolvido como evolução do sistema Login Seguro. O painel permite que o responsável pelos chamados acompanhe o andamento do atendimento, visualize a situação e as considerações técnicas de cada chamado e identifique informações relevantes da operação para apoiar a tomada de decisões.

A solução é composta por duas partes integradas: uma API REST, que consulta os dados no MongoDB Atlas e calcula os indicadores, e uma página web, que consome essa API e apresenta os dados em cartões, gráficos, ranking e tabela. A autenticação, os perfis de acesso e o layout foram reaproveitados do Login Seguro, cuja documentação é citada nas referências.

O código fonte está disponível no repositório: https://github.com/legferreira/login_completo_spring_boot. Ao final, é apresentada uma adaptação da mesma estrutura para um sistema de pedidos de pizzaria, disponível em: https://github.com/legferreira/pizzaria_dashboard_spring_boot. Essa adaptação é um exemplo prático da versatilidade dessa estrutura.

2 REQUISITOS FUNCIONAIS

2.1 Requisitos funcionais de usuário (por tela)

Tela de Login

- RF01: o gestor pode entrar com e-mail e senha
- RF02: o gestor pode encerrar a sessão (logout)

Painel do Gestor (Dashboard)

- RF03: o gestor pode visualizar os indicadores gerais: total de chamados, abertos, em andamento, concluídos e tempo médio de atendimento
- RF04: o gestor pode visualizar o gráfico de chamados atendidos por período
- RF05: o gestor pode visualizar a situação dos chamados em um gráfico por status

- RF06: o gestor pode visualizar o ranking dos tipos de chamados mais atendidos
- RF07: o gestor pode visualizar a lista de chamados com as observações técnicas
- RF08: o gestor pode filtrar os dados por período de abertura, status, tipo e técnico
- RF09: o gestor pode limpar os filtros e voltar a ver todos os chamados
- RF10: o gestor pode atualizar os dados do painel a qualquer momento

Página de Usuários

- RF11: o administrador pode alterar o perfil de um usuário para Gestor, liberando o acesso ao painel ou até excluir um perfil

2.2 Requisitos funcionais de sistema

- RF12: o sistema deve disponibilizar uma API REST com os dados dos chamados e os indicadores do painel
- RF13: o sistema deve permitir cadastrar e atualizar chamados pela API
- RF14: o sistema deve calcular os indicadores a cada consulta, a partir dos chamados filtrados, sem armazená-los no banco
- RF15: o sistema deve validar os campos obrigatórios (descrição, status e tipo) e retornar uma mensagem de erro quando não forem informados
- RF16: o sistema deve preencher a data de abertura quando não informada e a data de conclusão quando o status for “Concluído”
- RF17: o sistema deve atualizar o painel automaticamente a cada minuto
- RF18: o sistema deve exibir uma mensagem quando não houver chamados para os filtros informados ou quando a API não responder
- RF19: o sistema deve recusar o acesso ao painel e à API para usuários sem perfil Gestor ou Administrador

3 REQUISITOS NÃO FUNCIONAIS

Segurança

- RNF01: senhas protegidas com hash BCrypt
- RNF02: acesso ao painel e à API validado no servidor pelo Spring Security, e não apenas ocultado no menu
- RNF03: string de conexão com o banco e senha do administrador armazenadas em arquivo .env, fora do código e do GitHub

Interoperabilidade

- RNF04: API no padrão REST, com respostas em JSON e códigos HTTP de sucesso e de erro
- RNF05: datas enviadas e recebidas pela API no formato AAAA-MM-DD

Usabilidade

- RNF06: interface construída com Thymeleaf e Bootstrap, mantendo o layout do Login Seguro
- RNF07: datas e valores exibidos no padrão brasileiro

Persistência

- RNF08: dados armazenados no MongoDB Atlas, na coleção chamados

Manutenibilidade

- RNF09: separação de camadas (Controller, Service, Repository), com as regras de negócio isoladas nos services
- RNF10: versionamento com Git seguindo o fluxo Gitflow

Escalabilidade

- RNF11: o sistema deve suportar novos domínios, como a adaptação para pizzaria, sem exigir alteração na estrutura de autenticação

4 MODELAGEM DE DADOS

Os chamados são armazenados na coleção chamados do banco loginseguro, no MongoDB Atlas. Há uma entidade “Chamado”, que foi baseada no modelo do projeto de exemplo Thindesk, com o acréscimo das datas e da observação técnica, que são necessárias para os indicadores por período e para o acompanhamento do atendimento. Os campos são:

- **descricao:** texto que descreve o problema (obrigatório).
- **status:** Aberto, Em andamento, Concluído ou Cancelado (obrigatório).
- **tipo:** categoria do chamado, como Manutenção ou Instalação (obrigatório).
- **tecnico:** responsável pelo atendimento.
- **usuario:** quem abriu o chamado.
- **observacaoTecnica:** considerações técnicas sobre a solução.
- **dataAbertura:** data de abertura, preenchida automaticamente quando não informada.
- **dataConclusao:** data de conclusão, preenchida automaticamente quando o status passa para “Concluído”.

Os indicadores não são gravados no banco, eles são calculados pela API a cada consulta, a partir dos chamados filtrados, o que garante que o painel sempre reflita os dados atuais.

5 DOCUMENTAÇÃO DA API

5.1 Autenticação

Todos os endpoints exigem um usuário autenticado com perfil Gestor ou Administrador. No navegador, é usada a sessão do login. Em ferramentas externas, como o Postman, é usada autenticação Basic, com o e-mail e a senha do usuário.

5.2 Endpoints

Quadro 1 – Endpoints da API

Método	Endpoint	Descrição
GET	`/api/chamados`	Lista os chamados
GET	`/api/chamados/{id}`	Busca um chamado pelo id
POST	`/api/chamados`	Cadastra um chamado
PUT	`/api/chamados/{id}`	Atualiza um chamado
GET	`/api/dashboard/indicadores`	Indicadores gerais
GET	`/api/dashboard/atendidos-por-periodo`	Concluídos por data
GET	`/api/dashboard/status`	Quantidade por status
GET	`/api/dashboard/tipos`	Ranking dos tipos

Fonte: elaboração própria (2026).

5.3 Parâmetros de filtro

Os endpoints de consulta (GET /api/chamados e todos os /api/dashboard) aceitam os parâmetros opcionais a seguir, que podem ser combinados:

- **inicio e fim:** data inicial e final de abertura, no formato AAAA-MM-DD.
- **status:** status do chamado.
- **tipo:** tipo do chamado.
- **tecnico:** nome do técnico.

Exemplo de requisição com filtros:

```
GET/api/dashboard/indicadores?inicio=2026-11-01&fim=2026-11-30&status=Aberto
```

5.4 Exemplos de requisição e resposta

Indicadores gerais, GET /api/dashboard/indicadores:

```
{
  "total": 2,
  "abertos": 1,
  "emAndamento": 0,
  "concluidos": 1,
  "tempoMedioDias": 2.0
}
```


Situação dos chamados, GET /api/dashboard/status:


```
{"Aberto": 1, "Concluído": 1}
```


Cadastro de chamado, POST /api/chamados, com o corpo:


```
{
  "descricao": "Monitor sem imagem",
  "status": "Aberto",
  "tipo": "Manutenção",
  "tecnico": "João",
  "usuario": "Carla"
}
```


Resposta com o código 201 (Created), contendo o chamado salvo com id e data de abertura:


```
{
  "id": "6735f1c2a9d4e81b2c3f4a5b",
  "descricao": "Monitor sem imagem",
  "status": "Aberto",
  "tipo": "Manutenção",
  "tecnico": "João",
  "usuario": "Carla",
  "observacaoTecnica": null,
```

```
  "dataAbertura": "2026-11-13",
  "dataConclusao": null
}
```

A atualização, PUT /api/chamados/{id}, recebe o mesmo corpo do cadastro. Quando o status é alterado para “Concluído”, a data de conclusão é preenchida automaticamente.

5.5 Possíveis erros

Os erros da API retornam um JSON com o código e a mensagem, como no exemplo:

```
{"status": 400, "erro": "Informe o tipo"}
```

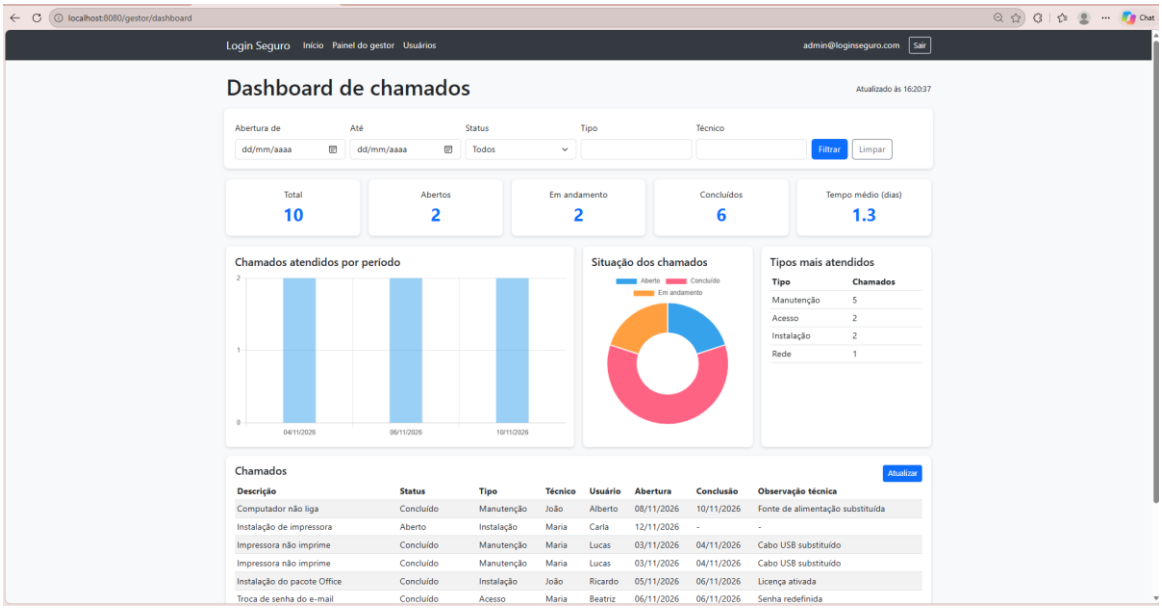
- **400 (Bad Request):** campo obrigatório vazio, JSON inválido ou data fora do formato AAAA-MM-DD.
- **401 (Unauthorized):** credenciais inválidas.
- **403 (Forbidden):** usuário autenticado sem perfil Gestor ou Administrador.
- **404 (Not Found):** chamado não encontrado para o id informado.

6 DASHBOARD ADMINISTRATIVO

O painel fica disponível no menu Painel do gestor, na rota /gestor/dashboard. Na parte superior ficam os filtros e os cartões de indicadores. Abaixo, o gráfico de chamados atendidos por período, o gráfico de situação e o ranking de tipos. Por fim, a tabela lista os chamados com as respectivas observações técnicas.

Os gráficos foram feitos com a biblioteca Chart.js, carregada por CDN, da mesma forma que o Bootstrap no projeto de exemplo da disciplina.

Figura 1 – Dashboard administrativo de chamados



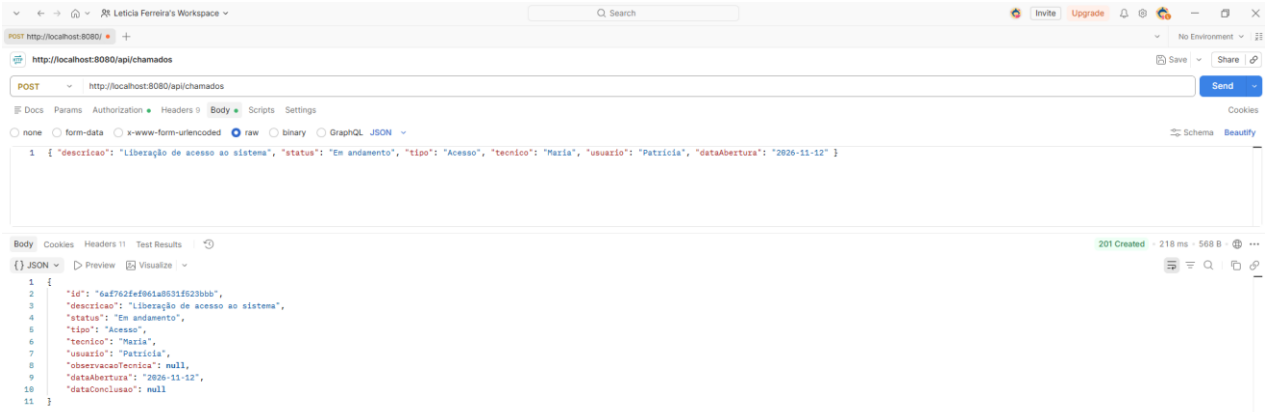
Fonte: elaboração própria (2026).

7 EVIDÊNCIAS DE TESTES

Os testes foram realizados manualmente, pelo navegador e pelo Postman, com os resultados a seguir:

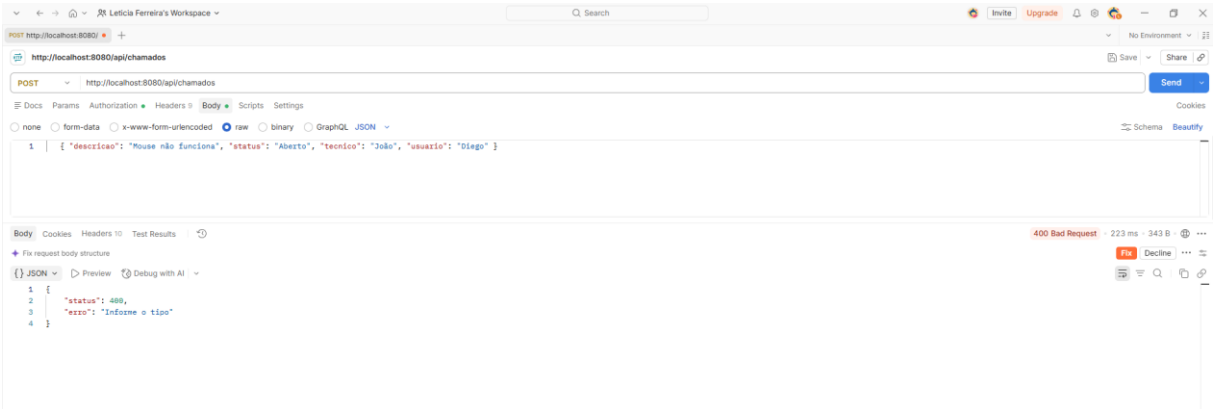
- **Cadastro pela API:** o chamado foi criado com o código 201 e, após clicar em Atualizar, o total de chamados, o gráfico de situação e o ranking de tipos foram atualizados no painel.
- **Atualização pela API:** ao alterar o status para “Concluído”, a data de conclusão foi preenchida e o chamado passou a aparecer no gráfico por período.
- **Validação de dados:** o envio sem o campo tipo retornou o código 400 com a mensagem "Informe o tipo".
- **Filtros:** ao filtrar por status, os indicadores, os gráficos e a tabela passaram a considerar apenas os chamados correspondentes. Sem resultados, o painel exibiu a mensagem de nenhum chamado encontrado.
- **Controle de acesso:** um usuário com perfil Usuário, ao acessar o painel, foi direcionado para a página de acesso negado.

Figura 2 – Cadastro de chamado pela API no Postman



Fonte: elaboração própria (2026).

Figura 3 – Erro de validação retornado pela API



Fonte: elaboração própria (2026).

8 ADAPTAÇÃO PARA SISTEMA DE PIZZARIA

Para demonstrar que a estrutura pode ser reaproveitada, o mesmo projeto foi adaptado para um sistema de pedidos de pizzaria, mantendo a autenticação, o layout, a organização em camadas e o formato da API. Essa adaptação foi publicada em um repositório separado:

https://github.com/legferreira/pizzaria_dashboard_spring_boot

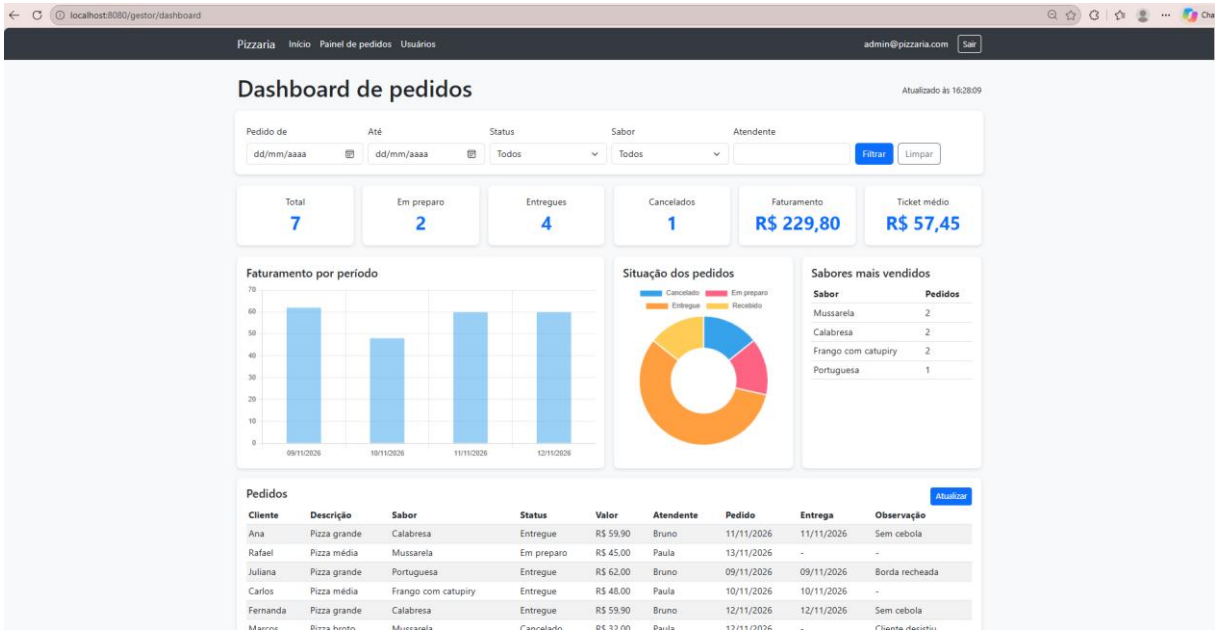
As principais mudanças foram:

- a entidade Chamado foi substituída pela entidade Pedido, com cliente, descrição, sabor, status, valor, atendente, observação e datas do pedido e da entrega;

- os perfis passaram a ser Administrador, Gestor e Funcionário;
- os indicadores passaram a mostrar pedidos em preparo, entregues, cancelados, faturamento e ticket médio;
- o gráfico por período passou a exibir o faturamento, e o ranking passou a exibir os sabores mais vendidos;
- os dados foram gravados em um banco separado, chamado pizzaria, no mesmo cluster do MongoDB Atlas.

A adaptação foi feita sem alterar a lógica de autenticação, o que confirma a proposta de um projeto genérico e reaproveitável.

Figura 4 – Dashboard da pizzaria



Fonte: elaboração própria (2026).

9 CONSIDERAÇÕES FINAIS

O dashboard desenvolvido atende aos requisitos propostos: apresenta indicadores, gráficos, ranking e filtros sobre os chamados, consome os dados por meio de uma API própria integrada ao MongoDB Atlas e trata a validação e a atualização dos dados. A reutilização da estrutura do Login Seguro e a adaptação para a pizzaria mostram que a separação em camadas facilita a evolução do sistema para novos contextos.

REFERÊNCIAS

CHART.JS. **Chart.js documentation**. Disponível em: https://www.chartjs.org/docs. Acesso em: 11 nov. 2026.

FERREIRA, Leticia Gabriela. **Login Seguro**: documentação do sistema. Mogi das Cruzes: Universidade de Mogi das Cruzes, 2026. Disponível em: https://github.com/legferreira/login_completo_spring_boot. Acesso em: 11 nov. 2026.

MONGODB. **MongoDB Atlas**. Disponível em: https://www.mongodb.com/products/platform. Acesso em: 11 nov. 2026.

SPRING. **Spring Boot**. Disponível em: https://spring.io/projects/spring-boot. Acesso em: 10 nov. 2026.

SPRING. **Spring Security**. Disponível em: https://spring.io/projects/spring-security. Acesso em: 10 nov. 2026.

THYMELEAF. **Tutorial: Thymeleaf + Spring**. Disponível em: https://www.thymeleaf.org. Acesso em: 10 nov. 2026.